ECx00G&EC600U&EG800G Series

QuecOpen(SDK) SPI NAND Flash API Reference Manual

LTE Standard Module Series

Version: 1.0

Date: 2023-09-07

Status: Released

At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.
Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China
Tel: +86 21 5108 6236
Email: info@quectel.com

Or our local offices. For more information, please visit:
http://www.quectel.com/support/sales.htm.

For technical support, or to report documentation errors, please visit:
http://www.quectel.com/support/technical.htm.
Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

About the Document

Revision History

Version	Date	Author	Description
-	2023-08-02	Ryan YI/ Sum LI	Creation of the document
1.0	2023-09-07	Ryan YI/ Sum LI	First official release

Contents

Table Index

Figure Index

1 Introduction

Quectel ECx00G family, EC600U series and EG800G series modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see ***document [1]***.

This document is applicable to QuecOpen® solution based on CSDK build environment. It outlines SPI NAND flash model adaption, general (4-wire) and dedicated (6-wire) SPI NAND flash APIs, SPI NAND flash file system mount API, as well as related examples on Quectel ECx00G family, EC600U series and EG800G series modules.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC600U Series
ECx00G	EC200G-CN
	EC800G-CN
	EC600G Series
-	EG800G Series

NOTE

In this document, SPI consists of Standard SPI, Dual SPI, and Quad SPI modes, where Standard SPI is general SPI (4-wire) and Dual/Quad SPI is dedicated SPI (6-wire). The controllers for general SPI and dedicated SPI are different and require corresponding driver codes. Pins defined by three modes are:

- Standard SPI: SCLK, CS#, SI and SO;
- Dual SPI: SCLK, CS#, SIO0 and SIO1;
- Quad SPI: SCLK, CS#, SIO0, SIO1, SIO2 and SIO3.

2 SPI NAND Flash Model Adaptation

quec_spi_nand_flash_prop.c and *quec_spi_nand_flash_prop.h*, NAND flash configuration files, are provided in QuecOpen CSDK, at the path *\components\ql_kernel\drivers*. You can add the specific flash model in configuration files.

This Chapter presents NAND flash model parameters and instructions for adding a flash model.

2.1. Supported NAND Flash Models

Parameters of some supported NAND flash models are defined within the *quec_spi_nand_flash_props* in *quec_spi_nand_flash_prop.c*, for example:

```
/* 4-line SPI NAND Flash supported model list,can add by yourself*/
static quec_spi_nand_flash_info_s quec_spi_nand_flash_props[] =
{
//You can cut out it by defining macros
//#define DISABLE_W25N02JWXXIF
#ifndef DISABLE_W25N02JWXXIF
    {
        // W25N02JWxxIF/IC  MID=EFBF22,2--2G
        .page_totalsize = 2112,
        .page_mainsize  = 2048,
        .page_sparesize = 64,
        .block_pagenum  = 64,
        .block_totalnum = 2048,
        .cache_blocknum = 5,
        {
            .page_spare_shift    = 0 ,//=page_mainsize = 2048
            .block_postion_shift = 2 ,//2050
            .block_type_shift    = 3 ,//2051
            .logic_addr_shift    = 16,//2064
            .page_used_shift     = 32,//2080
            .page_garbage_shift  = 33,//2081
            .page_num_shift      = 34,//2082
        },
        .nand_id        = 0xEF0F,      //bit[7:4]don't compare,should be 0
        .nand_id_type   = QUEC_NAND_FLASH_ID_24BIT,
    },
#endif
#ifndef DISABLE_GD5F1GQ5XEXXG
    {
        // GD5F1GQ5xExxG  MID=C841/C851,1--1G
        .page_totalsize = 2112,
        .page_mainsize  = 2048,
        .page_sparesize = 64,
        .block_pagenum  = 64,
        .block_totalnum = 1024,
        .cache_blocknum = 5,
        {
            .page_spare_shift    = 0 ,//=page_mainsize = 2048
            .block_postion_shift = 2 ,//2050
            .block_type_shift    = 3 ,//2051
            .logic_addr_shift    = 16,//2064
            .page_used_shift     = 32,//2080
            .page_garbage_shift  = 33,//2081
            .page_num_shift      = 34,//2082
        },
        .nand_id        = 0xC801,     //bit[7:4]don't compare,should be 0
        .nand_id_type   = QUEC_NAND_FLASH_ID_16BIT,
    }
#endif
};
```

Figure 1: Parameters of Supported NAND Flash Models

2.2. NAND Flash-Related Parameters

2.2.1. quec_spi_nand_flash_info_s

The *quec_spi_nand_flash_props* (whose type *quec_spi_nand_flash_info_s* is defined in *quec_spi_nand_flash_prop.h*) contains the parameters related to the NAND flash model as follows:

```
typedef struct
{
unsigned int page_totalsize;
unsigned int page_mainsize;
unsigned int page_sparesize;
unsigned int block_pagenum;
unsigned int block_totalnum;
unsigned int cache_blocknum;
quec_spi_nand_spare_info_s spare_info;
unsigned int nand_id;
quec_spi_nand_flash_id_type_e nand_id_type;
} quec_spi_nand_flash_info_s
```

- **Parameter**

Type	Parameter	Description
unsigned int	*page_totalsize*	Total size of each page (= size of main area + size of spare area). Unit: byte.
unsigned int	*page_mainsize*	Size of main area per page. Unit: byte.
unsigned int	*page_sparesize*	Size of spare area per page. Unit: byte.
unsigned int	*block_pagenum*	Block page count.
unsigned int	*block_totalnum*	Flash block count.
unsigned int	*cache_blocknum*	Cache block count. This value does not need to be modified.
quec_spi_nand_spare_info_s	*spare_info*	Configuration parameters of NAND flash spare area. See ***Chapter 2.2.1.1*** for details.
unsigned int	*nand_id*	NAND ID. See ***Chapter 2.2.1.3*** for details.
quec_spi_nand_flash_id_type_e	*nand_id_type*	Type of JEDEC ID. See ***Chapter 2.2.1.2*** for details.

2.2.1.1.quec_spi_nand_spare_info_s

The structure of configuration parameters of NAND flash spare area:

```
typedef struct
{
unsigned short page_spare_shift;
unsigned short block_postion_shift;
      unsigned short block_type_shift;
      unsigned short logic_addr_shift;
      unsigned short page_used_shift;
      unsigned short page_garbage_shift;
      unsigned short page_num_shift;
} quec_spi_nand_flash_info_s
```

- **Member**

Type	Parameter	Description
unsigned short	*page_spare_shift*	Offset address of bad block mark in spare area, 2 bytes.
unsigned short	*block_postion_shift*	Offset address of cache block mark in spare area, 1 byte.
unsigned short	*block_type_shift*	Offset address of block type mark in spare area, 1 byte.
unsigned short	*logic_addr_shift*	Offset address of logic block address mark in spare area, 4 bytes.
unsigned short	*page_used_shift*	Offset address of used page mark in spare area, 1 byte.
unsigned short	*page_garbage_shift*	Offset address of page recycling mark in spare area, 1 byte.
unsigned short	*page_num_shift*	Offset address of page number mark in spare area, 2 byte.

NOTE

1. Hardware ECC verification is enabled by default (software ECC verification is not supported). Select the field without ECC protection to configure the field value in the structure *quec_spi_nand_flash_info_s*. Otherwise, ECC verification reports an error. Note that the size and location of **the segment without ECC protection** in the spare area vary significantly for different NAND flash models. Therefore, it must be ensured that the segment without ECC protection occupies at least 12 bytes.
2. *page_spare_shift* is the field that indicates bad block. When setting the offset address of bad block mark in spare area, it must match the field for marking bad block specified in the NAND flash datasheet. Other fields can be adjusted as needed, but **the addresses occupied by each multibyte field must be consecutive**.

2.2.1.2.quec_spi_nand_flash_id_type_e

The enumeration of the JEDEC ID type:

```
typedef enum
{
    QUEC_NAND_FLASH_ID_16BIT,
    QUEC_NAND_FLASH_ID_24BIT,
}quec_spi_nand_flash_id_type_e
```

- **Member**

Member	Description
QUEC_NAND_FLASH_ID_16BIT	16-bit JEDEC ID, which is read with **9Fh** command and consists of manufacturer ID (8-bit) and device ID (8-bit).
QUEC_NAND_FLASH_ID_24BIT	24-bit JEDEC ID, which is read with **9Fh** command and consists of manufacturer ID (8-bit) and device ID (16-bit).

2.2.1.3.NAND ID

NAND ID is an additional distinct ID that is used only to differentiate between NAND flash models, and is separate from the JEDEC ID described above.

For different sub-models under a main model, JEDEC IDs may exhibit slight variations. However, since the primary focus is on distinguishing the main model in actual applications, the NAND ID is used instead of JEDEC ID to differentiate between NAND flash models. NAND flash IDs are converted to NAND IDs as follows:

Extract the high 16 bits of a 16-bit or 24-bit JEDEC ID, and set all bits [7:4] in the 16-bit segment to 0.

For example, if the W25N02JW JEDEC ID is 24-bit and read by *ql_spi_nand_read_devid_ex()* (see ***Chapter 3.1.3.4*** for details) as EFBF22, extract the high 16 bits, and set all bits [7:4] to 0, resulting in a NAND ID of 0xEF0F. Similarly, for the GD5F1GQ5 JEDEC ID, whether it is 0xC841 or 0xC851, convert it to NAND ID, and the value will be 0xC801.

2.3. Obtain and Configure Parameters

This section takes GD5F1GQ5 as an example to introduce how to obtain and configure the new NAND flash model information.

1. Refer to the NAND flash datasheet of GD5F1GQ5 to obtain the required configuration parameters.

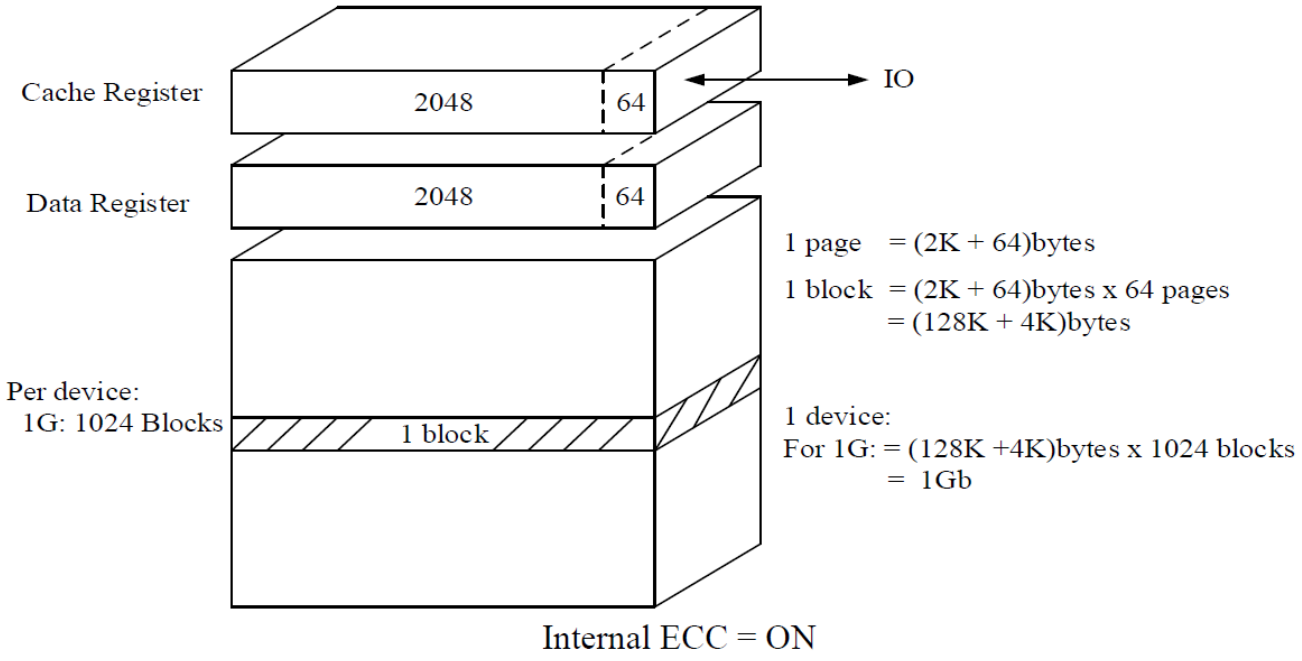

Figure 2: Array Organization of GD5F1GQ5

As can be seen from the figure above:

page_totalsize = 2 KB + 64 B= 2112 B
page_mainsize = 2 KB = 2048 B
page_sparesize = 64 B
block_pagenum = 64
block_totalnum = 1024
cache_blocknum = 5 (This parameter is fixed to 5 and does not need to be modified.)

Main Area(2KB)				Spare Area(128B)							
User data				User meta data(I+II)				ECC Parity Data			
Main0	Main1	Main2	Main3	Spare0	Spare1	Spare2	Spare3	Spare0	Spare1	Spare2	Spare3
(512B)	(512B)	(512B)	(512B)	(4B+12B)	(4B+12B)	(4B+12B)	(4B+12B)	(16B)	(16B)	(16B)	(16B)

Max Byte Address	Min Byte Address	ECC Protected	Area	Description
1FFH	000H	Yes	Main 0	User data 0
3FFH	200H	Yes	Main 1	User data 1
5FFH	400H	Yes	Main 2	User data 2
7FFH	600H	Yes	Main 3	User data 3
803H	800H	No	Spare 0	User meta 0 data I(1)
80FH	804H	Yes	Spare 0	User meta 0 data II
813H	810H	No	Spare 1	User meta 1 data I
81FH	814H	Yes	Spare 1	User meta 1 data II
823H	820H	No	Spare 2	User meta 2 data I
82FH	824H	Yes	Spare 2	User meta 2 data II
833H	830H	No	Spare 3	User meta 3 data I
83FH	834H	Yes	Spare 3	User meta 3 data II
84FH	840H	Yes	Spare 0	ECC Parity Data
85FH	850H	Yes	Spare 1	ECC Parity Data
86FH	860H	Yes	Spare 2	ECC Parity Data
87FH	870H	Yes	Spare 3	ECC Parity Data

Figure 3: ECC Protection and Spare Area of GD5F1GQ5

According to the distribution of segments without ECC protection and spare area in the above figure, the following parameters can be obtained:

page_spare_shift = 0
block_postion_shift = 2
block_type_shift = 3
logic_addr_shift = 16
page_used_shift = 32
page_garbage_shift = 33
page_num_shift = 34

For how to get *nand_id_type* and *nand_id*, see ***Chapters 2.2.1.2*** and ***2.2.1.3***.

2. Refer to ***Figure 1*** and add the new NAND flash model information in *quec_spi_nand_flash_props* in *quec_spi_nand_flash_prop.c*.

3 SPI NAND Flash APIs

3.1. General and Dedicated SPI NAND Flash APIs

3.1.1. Header File

ql_api_spi_nand_flash.h, the header file of general and dedicated SPI NAND flash APIs, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all header files mentioned in this document are in this directory.

3.1.2. API Overview

Table 2: API Overview

Function	Description
ql_spi_nand_init()	Initializes general SPI (SPI bus and clock frequency only).
ql_spi_nand_init_ext()	Initializes general SPI.
ql_spi_nand_read_devid()	Reads general SPI NAND flash JEDEC ID.
ql_spi_nand_read_devid_ex()	Reads general SPI NAND flash JEDEC ID according to JEDEC ID type.
ql_spi_nand_read_page_spare()	Reads both the main data area and the spare area from a specific page in general SPI NAND flash.
ql_spi_nand_write_page_spare()	Writes data to both the main data area and the spare area of a specific page in general SPI NAND flash.
ql_spi_nand_write_spare()	Writes data to the spare area of general SPI NAND flash.
ql_spi_nand_read_status()	Reads the value of general SPI NAND flash status register.
ql_spi_nand_write_status()	Writes a value to general SPI NAND flash status register.
ql_spi_nand_erase_block()	Erases a block of general SPI NAND flash.
ql_spi_nand_reset()	Resets general SPI NAND flash.

ql_spi6_nand_init()	Initializes dedicated SPI (SPI bus, clock frequency and frequency division coefficient only).
ql_spi6_nand_init_ext()	Initializes dedicated SPI.
ql_spi6_nand_read_devid()	Reads dedicated SPI NAND flash JEDEC ID.
ql_spi6_nand_read_devid_ex()	Reads dedicated SPI NAND flash JEDEC ID according to JEDEC ID type.
ql_spi6_nand_read_page_spare()	Reads both the main data area and the spare area from a specific page in a dedicated SPI NAND flash.
ql_spi6_nand_write_page_spare()	Writes data to both the main data area and the spare area of a specific page in a dedicated SPI NAND flash.
ql_spi6_nand_write_spare()	Writes data to the spare area of dedicated SPI NAND flash.
ql_spi6_nand_read_status()	Reads a value of dedicated SPI NAND flash status register.
ql_spi6_nand_write_status()	Writes a value to dedicated SPI NAND flash status register.
ql_spi6_nand_erase_block()	Erases a block of dedicated SPI NAND flash.
ql_spi6_nand_reset()	Resets dedicated SPI NAND flash.

3.1.3. API Description

3.1.3.1.ql_spi_nand_init

This function initializes general SPI (SPI bus and clock frequency only). Before calling this function, you need to set relevant GPIO pins to general SPI function by *ql_pin_set_func()*. See ***document [2]*** for details.

Calling *ql_spi_nand_init()* will configure *ql_spi_input_sel_e* to *QL_SPI_DI_1*, *ql_spi_transfer_mode_e* to *QL_SPI_DIRECT_POLLING*, and *ql_spi_cs_sel_e* to *QL_SPI_CS0* by default. See ***Chapters 3.1.3.1.4***, ***3.1.3.1.5***, and ***3.1.3.1.6*** for details.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_init(ql_spi_port_e port, ql_spi_clk_e spiclk)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

spiclk:
[In] SPI clock frequency. See ***Chapter 3.1.3.1.3*** for details.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

NOTE

Selecting a wrong SPI bus for initialization can lead to failures in reading, writing, or erasing NAND flash. Therefore, you need to check the initialization return value first. If initialization fails, please do not perform read, write, and erase operations on the NAND flash.

3.1.3.1.1. ql_errcode_spi_nand_e

The enumeration of result codes of SPI NAND flash API:

```
typedef ql_errcode_spi_flash_e ql_errcode_spi_nand_e
```

```
typedef enum
{
    QL_SPI_FLASH_SUCCESS                    =    0,
    QL_SPI_FLASH_ERROR                      =    1 |
(QL_COMPONENT_STORAGE_EXTFLASH << 16),
    QL_SPI_FLASH_PARAM_TYPE_ERROR,
    QL_SPI_FLASH_PARAM_DATA_ERROR,
    QL_SPI_FLASH_PARAM_ACQUIRE_ERROR,
    QL_SPI_FLASH_PARAM_NULL_ERROR,
    QL_SPI_FLASH_DEV_NOT_ACQUIRE_ERROR,
    QL_SPI_FLASH_PARAM_LENGTH_ERROR,
    QL_SPI_FLASH_MALLOC_MEM_ERROR,
    QL_SPI_FLASH_NOT_FLASH_CONN_ERROR,
    QL_SPI_FLASH_NOT_SUPPORT_ERROR,
    QL_SPI_FLASH_OP_NOT_SUPPORT_ERROR,
    QL_SPI_FLASH_ECC_ERROR,
    QL_SPI_FLASH_PROGRAM_ERROR,
    QL_SPI_FLASH_ERASE_ERROR,
    QL_SPI_FLASH_MUTEX_CREATE_ERROR,
    QL_SPI_FLASH_MUTEX_LOCK_ERROR,
    QL_SPI_FLASH_SET_GPIO_ERROR
}ql_errcode_spi_flash_e;
```

- **Member**

Member	Description
QL_SPI_FLASH_SUCCESS	Successful execution
QL_SPI_FLASH_ERROR	Other errors related to SPI bus
QL_SPI_FLASH_PARAM_TYPE_ERROR	Parameter type error
QL_SPI_FLASH_PARAM_DATA_ERROR	Parameter data error
QL_SPI_FLASH_PARAM_ACQUIRE_ERROR	Unable to get parameters
QL_SPI_FLASH_PARAM_NULL_ERROR	Parameter NULL error
QL_SPI_FLASH_DEV_NOT_ACQUIRE_ERROR	Unable to get SPI bus
QL_SPI_FLASH_PARAM_LENGTH_ERROR	Parameter length error
QL_SPI_FLASH_MALLOC_MEM_ERROR	Memory allocation error
QL_SPI_FLASH_NOT_FLASH_CONN_ERROR	Flash chip not connected
QL_SPI_FLASH_NOT_SUPPORT_ERROR	Flash model not supported (for external NOR flash only)
QL_SPI_FLASH_OP_NOT_SUPPORT_ERROR	Operation not supported by the flash model
QL_SPI_FLASH_ECC_ERROR	NAND flash ECC error
QL_SPI_FLASH_PROGRAM_ERROR	Flash program error
QL_SPI_FLASH_ERASE_ERROR	Flash erase error
QL_SPI_FLASH_MUTEX_CREATE_ERROR	Flash mutex creation error
QL_SPI_FLASH_MUTEX_LOCK_ERROR	Flash mutex locking error
QL_SPI_FLASH_SET_GPIO_ERROR	Flash setting GPIO error

3.1.3.1.2. ql_spi_port_e

The enumeration of SPI bus:

```
typedef enum
{
    QL_SPI_PORT1,
    QL_SPI_PORT2,
}ql_spi_port_e;
```

- **Member**

Member	Description
QL_SPI_PORT1	SPI1 bus
QL_SPI_PORT2	SPI2 bus

3.1.3.1.3. ql_spi_clk_e

The enumeration of general SPI clock frequency of EC600U series module:

```
typedef enum
{
    QL_SPI_CLK_INVALID=-1,
    QL_SPI_CLK_781_25KHZ = 781250,
    QL_SPI_CLK_1_5625MHZ = 1562500,
    QL_SPI_CLK_3_125MHZ = 3125000,
    QL_SPI_CLK_5MHZ = 5000000,
    QL_SPI_CLK_6_25MHZ = 6250000,
    QL_SPI_CLK_10MHZ = 10000000,
    QL_SPI_CLK_12_5MHZ = 12500000,
    QL_SPI_CLK_20MHZ = 20000000,
    QL_SPI_CLK_25MHZ = 25000000,
    QL_SPI_CLK_33_33MHZ = 33000000,
    QL_SPI_CLK_50MHZ_MAX = 50000000,
}ql_spi_clk_e;
```

- **Member**

Member	Description
QL_SPI_CLK_INVALID	Invalid parameter
QL_SPI_CLK_781_25KHZ	Clock frequency: 781.25 kHz
QL_SPI_CLK_1_5625MHZ	Clock frequency: 1.5625 MHz
QL_SPI_CLK_3_125MHZ	Clock frequency: 3.125 MHz
QL_SPI_CLK_5MHZ	Clock frequency: 5 MHz
QL_SPI_CLK_6_25MHZ	Clock frequency: 6.25 MHz
QL_SPI_CLK_10MHZ	Clock frequency: 10 MHz
QL_SPI_CLK_12_5MHZ	Clock frequency: 12.5 MHz

QL_SPI_CLK_20MHZ	Clock frequency: 20 MHz
QL_SPI_CLK_25MHZ	Clock frequency: 25 MHz
QL_SPI_CLK_33_33MHZ	Clock frequency: 33.33 MHz
QL_SPI_CLK_50MHZ_MAX	Clock frequency: 50 MHz (Maximum)

The enumeration of general SPI clock frequency of ECx00G family and EG800G series modules:

```
typedef enum
{
    QL_SPI_CLK_INVALID=-1,
    QL_SPI_CLK_97_656KHZ_MIN = 97656,
    QL_SPI_CLK_100KHZ = 100000,
    QL_SPI_CLK_812_5KHZ = 812500,
    QL_SPI_CLK_1_3MHZ = 1300000,
    QL_SPI_CLK_1_625MHZ = 1625000,
    QL_SPI_CLK_1_5625MHZ = QL_SPI_CLK_1_625MHZ,
    QL_SPI_CLK_2MHZ = 2000000,
    QL_SPI_CLK_3_25MHZ = 3250000,
    QL_SPI_CLK_4_333MHZ = 4333000,
    QL_SPI_CLK_6_6MHZ = 6600000,
    QL_SPI_CLK_11_93MHZ = 11930000,
    QL_SPI_CLK_13MHZ = 13000000,
    QL_SPI_CLK_13_92MHZ = 13920000,
    QL_SPI_CLK_16_7MHZ = 16700000,
    QL_SPI_CLK_20_875MHZ = 20875000,
    QL_SPI_CLK_27_83MHZ = 27830000,
    QL_SPI_CLK_25MHZ = QL_SPI_CLK_27_83MHZ,
    QL_SPI_CLK_41_75MHZ = 41750000,
    QL_SPI_CLK_50MHZ_MAX = 50000000,
}ql_spi_clk_e
```

- **Member**

Member	Description
QL_SPI_CLK_INVALID	Invalid parameter
QL_SPI_CLK_97_656KHZ_MIN	Clock frequency: 97.656 kHz
QL_SPI_CLK_100KHZ	Clock frequency: 100 kHz
QL_SPI_CLK_812_5KHZ	Clock frequency: 812.5 kHz

QL_SPI_CLK_1_3MHZ	Clock frequency: 1.3 MHz
QL_SPI_CLK_1_625MHZ	Clock frequency: 1.625 MHz
QL_SPI_CLK_1_5625MHZ	Clock frequency: 1.625 MHz
QL_SPI_CLK_2MHZ	Clock frequency: 2 MHz
QL_SPI_CLK_3_25MHZ	Clock frequency: 3.25 MHz
QL_SPI_CLK_4_333MHZ	Clock frequency: 4.333 MHz
QL_SPI_CLK_6_6MHZ	Clock frequency: 6.6 MHz
QL_SPI_CLK_11_93MHZ	Clock frequency: 11.93 MHz
QL_SPI_CLK_13MHZ	Clock frequency: 13 MHz
QL_SPI_CLK_13_92MHZ	Clock frequency: 13.92 MHz
QL_SPI_CLK_16_7MHZ	Clock frequency: 16.7 MHz
QL_SPI_CLK_20_875MHZ	Clock frequency: 20.875 MHz
QL_SPI_CLK_27_83MHZ	Clock frequency: 27.83 MHz
QL_SPI_CLK_25MHZ	Clock frequency: 27.83 MHz
QL_SPI_CLK_41_75MHZ	Clock frequency: 41.75 MHz
QL_SPI_CLK_50MHZ_MAX	Clock frequency: 50 MHz (Maximum)

3.1.3.1.4. ql_spi_input_sel_e

The enumeration of data input pin of general SPI:

```
typedef enum
{
    QL_SPI_DI_0 = 0,
    QL_SPI_DI_1,
    QL_SPI_DI_2,
}ql_spi_input_sel_e;
```

- **Member**

Member	Description
QL_SPI_DI_0	DI0 as data input pin (currently not supported)

QL_SPI_DI_1	DI1 as data input pin
QL_SPI_DI_2	DI2 as data input pin (currently not supported)

3.1.3.1.5. ql_spi_transfer_mode_e

The enumeration of general SPI transfer mode:

```
typedef enum
{
    QL_SPI_DIRECT_POLLING = 0,
    QL_SPI_DIRECT_IRQ,
    QL_SPI_DMA_POLLING,
    QL_SPI_DMA_IRQ,
}ql_spi_transfer_mode_e;
```

- **Member**

Member	Description
QL_SPI_DIRECT_POLLING	FIFO read/write with polling waiting.
QL_SPI_DIRECT_IRQ	FIFO read/write with interrupt notification. (Currently not supported.)
QL_SPI_DMA_POLLING	DMA read/write with polling waiting.
QL_SPI_DMA_IRQ	DMA read/write with interrupt notification. (Currently not supported.)

NOTE

When EC600U series module uses DMA polling mode, general SPI and SD card preempt DMA channels. If the SD card already uses DMA channels, general SPI initialization will fail. This does not apply to ECx00G family and EG800G series modules.

3.1.3.1.6. ql_spi_cs_sel_e

The enumeration of general CS pin:

```
typedef enum
{
    QL_SPI_CS0 = 0,
    QL_SPI_CS1,
    QL_SPI_CS2,
    QL_SPI_CS3,
```

```
}ql_spi_cs_sel_e;
```

- **Member**

Member	Description
QL_SPI_CS0	CS0 as general SPI CS pin
QL_SPI_CS1	CS1 as general SPI CS pin
QL_SPI_CS2	CS2 as general SPI CS pin (currently not supported)
QL_SPI_CS3	CS3 as general SPI CS pin (currently not supported)

3.1.3.2.ql_spi_nand_init_ext

This function initializes general SPI, including SPI bus, clock frequency, transfer mode, data input pin, and CS pin. Before calling this function, you need to set relevant GPIO pins to general SPI function by *ql_pin_set_func()*. See ***document [2]*** for details.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_init_ext(ql_spi_nand_config_s nand_config)
```

- **Parameter**

nand_config:
[In] General SPI configuration parameters. See ***Chapter 3.1.3.2.1*** for details.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

NOTE

Selecting a wrong SPI bus for initialization can lead to failures in reading, writing, or erasing NAND flash. Therefore, you need to check the initialization return value first. If initialization fails, please do not perform read, write, and erase operations.

3.1.3.2.1. ql_spi_nand_config_s

The structure of general SPI configuration parameters:

```
typedef ql_spi_flash_config_s ql_spi_nand_config_s
```

```
typedef struct
{
    ql_spi_port_e port;
    ql_spi_clk_e spiclk;
    ql_spi_input_sel_e input_sel;
    ql_spi_transfer_mode_e transmode;
    ql_spi_cs_sel_e cs;
} ql_spi_flash_config_s;
```

- **Parameter**

Type	Parameter	Description
ql_spi_port_e	*port*	SPI bus. See ***Chapter 3.1.3.1.2*** for details.
ql_spi_clk_e	*spiclk*	General SPI clock frequency. See ***Chapter 3.1.3.1.3*** for details.
ql_spi_input_sel_e	*input_sel*	General SPI data input pin. See ***Chapter 3.1.3.1.4*** for details.
ql_spi_transfer_mode_e	*transmode*	General SPI transfer mode. See ***Chapter 3.1.3.1.5*** for details.
ql_spi_cs_sel_e	*cs*	General SPI CS pin. See ***Chapter 3.1.3.1.6*** for details.

3.1.3.3.ql_spi_nand_read_devid

This function reads general SPI NAND flash JEDEC ID.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_read_devid(ql_spi_port_e port, unsigned char *mid)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

mid:
[Out] General SPI NAND flash JEDEC ID.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.4.ql_spi_nand_read_devid_ex

This function reads general SPI NAND flash JEDEC ID according to JEDEC ID type.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_read_devid_ex(ql_spi_port_e port, unsigned char *mid,ql_spi_nand_id_type_e mid_type)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

mid:
[Out] General SPI NAND flash JEDEC ID.

mid_type:
[In] NAND flash JEDEC ID type. See ***Chapter 3.1.3.4.1*** for details.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.4.1. ql_spi_nand_id_type_e

The enumeration of NAND flash JEDEC ID type:

```
typedef enum
{
    QL_NAND_FLASH_ID_16BIT = 0,,
    QL_NAND_FLASH_ID_24BIT,
}ql_spi_nand_id_type_e
```

- **Member**

Member	Description
QL_NAND_FLASH_ID_16BIT	16-bit JEDEC ID, which is read with **9Fh** command and consists of manufacturer ID (8-bit) and device ID (8-bit).
QL_NAND_FLASH_ID_24BIT	24-bit JEDEC ID, which is read with **9Fh** command and consists of manufacturer ID (8-bit) and device ID (16-bit).

3.1.3.5.ql_spi_nand_read_page_spare

This function reads both the main data area (2048 bytes) and spare area (64 bytes) from a specific page in a general SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_read_page_spare(ql_spi_port_e port, unsigned int page_addr, unsigned short column_addr, unsigned char *data, int len)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

page_addr:
[In] Page address.

column_addr:
[In] Column address.

data:
[Out] Pointer to read data.

len:
[In] Length of data to be read. Unit: byte.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.6.ql_spi_nand_write_page_spare

This function writes data to both the main data area (2048 bytes) and spare area (64 bytes) of a specific page in a general SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_write_page_spare(ql_spi_port_e port, unsigned int page_addr, unsigned short column_addr, unsigned char *data, int len)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

page_addr:
[In] Page address.

column_addr:
[In] Column address.

data:
[In] Data to be written.

len:
[In] Length of data to be written. Unit: byte.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.7.ql_spi_nand_write_spare

This function writes data to the spare area (64 bytes) of a general SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_write_spare(ql_spi_port_e port, unsigned int page_addr, unsigned char *data, int len)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

page_addr:
[In] Page address.

data:
[In] Data to be written.

len:
[In] Length of data to be written. Unit: byte.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.8.ql_spi_nand_read_status

This function reads the value of a general SPI NAND flash status register.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_read_status(ql_spi_port_e port, ql_spi_nand_status_reg_e reg, unsigned char *status)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

reg:
[In] Status register. See ***Chapter 3.1.3.8.1*** for details.

status:
[Out] Values read out from a status register.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.8.1. ql_spi_nand_status_reg_e

The enumeration of status register:

```
typedef enum
{
    QL_NAND_FLASH_STATUS_1 = 0,
    QL_NAND_FLASH_STATUS_2,
    QL_NAND_FLASH_STATUS_3,
    QL_NAND_FLASH_STATUS_4,
}ql_spi_nand_status_reg_e;
```

- **Member**

Member	Description
QL_NAND_FLASH_STATUS_1	Status register 1
QL_NAND_FLASH_STATUS_2	Status register 2
QL_NAND_FLASH_STATUS_3	Status register 3

QL_NAND_FLASH_STATUS_4	Status register 4

3.1.3.9.ql_spi_nand_write_status

This function writes a value to a general SPI NAND flash status register.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_write_status(ql_spi_port_e port, ql_spi_nand_status_reg_e reg, unsigned char status)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

reg:
[In] Status register. See ***Chapter 3.1.3.8.1*** for details.

status:
[In] Values to be written to a status register.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.10. ql_spi_nand_erase_block

This function erases a block of general SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_erase_block(ql_spi_port_e port, unsigned int page_addr)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

page_addr:
[In] First page address of a block.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.11. ql_spi_nand_reset

This function resets general SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi_nand_reset(ql_spi_port_e port)
```

- **Parameter**

port:
[In] SPI bus. See ***Chapter 3.1.3.1.2*** for details.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.12. ql_spi6_nand_init

This function initializes dedicated SPI (SPI bus, clock frequency and frequency division coefficient only). Before calling this function, you need to set relevant GPIO pins to dedicated SPI function by *ql_pin_set_func()*. See ***document [2]*** for details.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_init(ql_spi_port_e port, ql_spi6_nand_flash_clk_e clk, uint8_t clk_div)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

clk:
[In] SPI clock frequency. See ***Chapter 3.1.3.12.1*** for details.

clk_div:
[In] Frequency division coefficient. Range: 1–255.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

NOTE

It is necessary to check the return value of initialization. If the initialization fails, please do not perform read, write, and erase operations of NAND lash.

3.1.3.12.1.ql_spi6_nand_flash_clk_e

The enumeration of dedicated SPI clock frequency of EC600U series module:

```
typedef enum
{
    QL_SPI6_NAND_FLASH_CLK_SRC_62MHZ = 1,
    QL_SPI6_NAND_FLASH_CLK_SRC_82MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_124MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_142MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_166MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_182MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_200MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_222MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_250MHZ,
    QL_SPI6_NAND_FLASH_CLK_SRC_INVALID = 10,
}ql_spi6_nand_flash_clk_e
```

- **Member**

Member	Description
QL_SPI6_NAND_FLASH_CLK_SRC_62MHZ	Clock frequency: 62 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_82MHZ	Clock frequency: 82 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_124MHZ	Clock frequency: 124 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_142MHZ	Clock frequency: 142 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_166MHZ	Clock frequency: 166 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_182MHZ	Clock frequency: 182 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_200MHZ	Clock frequency: 200 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_222MHZ	Clock frequency: 222 MHz

QL_SPI6_NAND_FLASH_CLK_SRC_250MHZ	Clock frequency: 250 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_INVALID	Invalid parameters

The enumeration of dedicated SPI clock frequency of ECx00G family and EG800G series modules:

```
typedef enum
{
    QL_SPI6_NAND_FLASH_CLK_SRC_62MHZ = 0x1,
QL_SPI6_NAND_FLASH_CLK_SRC_71MHZ = 0x3,
    QL_SPI6_NAND_FLASH_CLK_SRC_83MHZ = 0x5,
    QL_SPI6_NAND_FLASH_CLK_SRC_91MHZ = 0x6,
    QL_SPI6_NAND_FLASH_CLK_SRC_100MHZ = 0x7,
    QL_SPI6_NAND_FLASH_CLK_SRC_111MHZ = 0x8,
    QL_SPI6_NAND_FLASH_CLK_SRC_125MHZ = 0x9,
    QL_SPI6_NAND_FLASH_CLK_SRC_142MHZ = 0xa,
    QL_SPI6_NAND_FLASH_CLK_SRC_166MHZ = 0xb,
    QL_SPI6_NAND_FLASH_CLK_SRC_200MHZ = 0xc,
    QL_SPI6_NAND_FLASH_CLK_SRC_500MHZ = 0xf,
    QL_SPI6_NAND_FLASH_CLK_SRC_INVALID = 0x10,
}ql_spi6_nand_flash_clk_e
```

- **Member**

Member	Description
QL_SPI6_NAND_FLASH_CLK_SRC_62MHZ	Clock source frequency: 62 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_71MHZ	Clock source frequency: 71 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_83MHZ	Clock source frequency: 83 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_91MHZ	Clock source frequency: 91 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_100MHZ	Clock source frequency: 100 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_111MHZ	Clock source frequency: 111 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_125MHZ	Clock source frequency: 125 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_142MHZ	Clock source frequency: 142 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_166MHZ	Clock source frequency: 166 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_200MHZ	Clock source frequency: 200 MHz
QL_SPI6_NAND_FLASH_CLK_SRC_500MHZ	Clock source frequency: 500 MHz

QL_SPI6_NAND_FLASH_CLK_SRC_INVALID	Invalid parameters

3.1.3.13. ql_spi6_nand_init_ext

This function initializes dedicated SPI, including SPI bus, clock frequency, frequency division coefficient, SPI mode, sampling delay coefficient, and whether to use a low speed clock. Before calling this function, you need to set relevant GPIO pins to dedicated SPI function by *ql_pin_set_func()*. See ***document [2]*** for details.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_init_ext(ql_spi6_nand_config_s config)
```

- **Parameter**

config:
[In] Dedicated SPI configuration parameters. See ***Chapter 3.1.3.13.1*** for details.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

NOTE

It is necessary to check the return value of initialization. If the initialization fails, please do not perform read, write, and erase operations of NAND flash.

3.1.3.13.1.ql_spi6_nand_config_s

The structure of dedicated SPI configuration parameters:

```
typedef ql_spi_flash_config_s ql_spi_nand_config_s
```

```
typedef struct
{
    ql_spi_port_e port;
    ql_spi6_nand_flash_clk_e clk;
    uint8_t   clk_div;
    uint8_t   quad_mode;
    uint8_t   sample_delay;
    uint8_t   isslow;
```

```
} ql_spi6_nand_config_s
```

- **Parameter**

Type	Parameter	Description
ql_spi_port_e	*port*	Dedicated SPI bus. This parameter is meaningless.
ql_spi6_nand_flash_clk_e	*clk*	Dedicated SPI clock frequency. See ***Chapter 3.1.3.12.1*** for details.
uint8_t	*clk_div*	Dedicated SPI frequency division coefficient. Range: 1–255.
uint8_t	*quad_mode*	Dedicated SPI mode. *1* Quad SPI *0* Dual SPI
uint8_t	*sample_delay*	Sampling delay coefficient. Range: 0–255.
uint8_t	*isslow*	Indicates whether to use a low speed clock. *1* Using a low speed clock *0* Using a fast speed clock (This parameter is only valid for EC600U series module.)

3.1.3.14. ql_spi6_nand_read_devid

This function reads dedicated SPI NAND flash JEDEC ID.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_read_devid(ql_spi_port_e port, unsigned char *mid)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

mid:
[Out] Dedicated SPI NAND flash JEDEC ID.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.15. ql_spi6_nand_read_devid_ex

This function reads dedicated SPI NAND flash JEDEC ID according to JEDEC ID type.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_read_devid_ex(ql_spi_port_e port, unsigned char *mid, ql_spi_nand_id_type_e mid_type)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

mid:
[Out] Dedicated SPI NAND flash JEDEC ID.

mid_type:
[In] NAND flash JEDEC ID type. See ***Chapter 3.1.3.4.1*** for details.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.16. ql_spi6_nand_read_page_spare

This function reads both the main data area (2048 bytes) and spare area (64 bytes) from a specific page in a dedicated SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_read_page_spare(ql_spi_port_e port, unsigned int page_addr, unsigned short column_addr, unsigned char *data, int len)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

page_addr:
[In] Page address.

column_addr:
[In] Column address.

data:
[Out] Pointer to read data.

len:
[In] Length of data to be read. Unit: byte.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.17. ql_spi6_nand_write_page_spare

This function writes data to both the main data area (2048 bytes) and spare area (64 bytes) of a specific page in a dedicated SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_write_page_spare(ql_spi_port_e port, unsigned int page_addr, unsigned short column_addr, unsigned char *data, int len)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

page_addr:
[In] Page address.

column_addr:
[In] Column address.

data:
[In] Data to be written.

len:
[In] Length of data to be written. Unit: byte.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.18. ql_spi6_nand_write_spare

This function writes data to the spare area (64 bytes) of a dedicated SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_write_spare(ql_spi_port_e port, unsigned int page_addr, unsigned char *data, int len)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

page_addr:
[In] Page address.

data:
[In] Data to be written.

len:
[In] Length of data to be written. Unit: byte.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.19. ql_spi6_nand_read_status

This function reads the value of a dedicated SPI NAND flash status register.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_read_status(ql_spi_port_e port, ql_spi_nand_status_reg_e reg, unsigned char *status)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

reg:
[In] Status register. See ***Chapter 3.1.3.8.1*** for details.

status:
[Out] Values read from a status register.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.20. ql_spi6_nand_write_status

This function writes a value to a dedicated SPI NAND flash status register.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_write_status(ql_spi_port_e port, ql_spi_nand_status_reg_e reg, unsigned char status)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

reg:
[In] Status register. See ***Chapter 3.1.3.8.1*** for details.

status:
[In] Values to be written to a status register.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.21. ql_spi6_nand_erase_block

This function erases a block of a dedicated SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_erase_block(ql_spi_port_e port, unsigned int page_addr)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

page_addr:
[In] First page address of a block.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.1.3.22. ql_spi6_nand_reset

This function resets a dedicated SPI NAND flash.

- **Prototype**

```
ql_errcode_spi_nand_e ql_spi6_nand_reset(ql_spi_port_e port)
```

- **Parameter**

port:
[In] SPI bus. This parameter is meaningless.

- **Return Value**

See ***Chapter 3.1.3.1.1*** for details.

3.2. SPI NAND Flash File System Mount API

3.2.1. Header File

ql_api_fs_nand_flash.h, the header file of SPI NAND flash file system mount API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all header files mentioned in this document are in this directory.

3.2.2. API Overview

Table 3: API Overview

Function	Description
ql_nand_flash_mkfs()	Formats SPI NAND flash to the FATFS format.
ql_nand_flash_umount()	Unmounts SPI NAND flash from the FATFS file system.
ql_nand_flash_mount()	Mounts SPI NAND flash to the FATFS file system.
ql_nand_get_hw_info()	Gets hardware information of SPI NAND flash.

3.2.3. API Description

3.2.3.1.ql_nand_flash_mkfs

This function formats SPI NAND flash to the FATFS format. After a successful formatting, reading NAND flash with the file system yields an empty result.

- **Prototype**

```
ql_errcode_nand_flash_e ql_nand_flash_mkfs(uint8_t opt)
```

- **Parameter**

opt:
[In] Target format after formatting.
0x01 FM_FAT (FAT12 or FAT16)
0x02 FM_FAT32

- **Return Value**

See ***Chapter 3.2.3.1.1*** for details.

NOTE

Select the file system format based on the block count. The number of blocks specified by the file system corresponds to the file system types as follows:

- If the block count is between 1024 and 8220, only FM_FAT (FAT12) is supported;
- If the block count is between 8229 and 4190430, FM_FAT (FAT16) can be used;
- If the block count is between 66130 and 67048580, FM_FAT32 can be used.

Therefore,

- If the block count is below 66130, the only format available is FM_FAT (FAT12 or FAT16);
- If the block count is between 66130 and 4190430, the available formats are FM_FAT32 or FM_FAT (FAT16);
- If the block count is greater than or equal to 41904301, the only available format is FM_FAT32.

For example, a 1 GB NAND flash with 60928 blocks can be formatted in FM_FAT (FAT12 or FAT16), while a 2 GB NAND flash with over 66130 blocks can be formatted in FM_FAT32 or FM_FAT (FAT16).

3.2.3.1.1. ql_errcode_nand_flash_e

The SPI NAND flash file system mount result code enumeration:

```
typedef enum
{
```

```
    QL_NAND_FLASH_SUCCESS                   = QL_SUCCESS,
    QL_NAND_FLASH_INVALID_PARAM_ERR         =
1|QL_NAND_FLASH_ERRCODE_BASE,
    QL_NAND_FLASH_MOUNT_ERR,
    QL_NAND_FLASH_MKFS_ERR,
    QL_NAND_FLASH_NOT_MOUNT,
    QL_NAND_FLASH_MUTEX_CREATE_ERR,
    QL_NAND_FLASH_MUTEX_LOCK_ERR,
} ql_errcode_nand_flash_e
```

- **Member**

Member	Description
QL_NAND_FLASH_SUCCESS	Successful execution
QL_NAND_FLASH_INVALID_PARAM_ERR	Invalid parameter
QL_NAND_FLASH_MOUNT_ERR	Flash mounting error
QL_NAND_FLASH_MKFS_ERR	Flash formatting error
QL_NAND_FLASH_NOT_MOUNT	Flash unmounting error
QL_NAND_FLASH_MUTEX_CREATE_ERR	Flash mutex creation error
QL_NAND_FLASH_MUTEX_LOCK_ERR	Flash mutex locking error

3.2.3.2.ql_nand_flash_umount

This function unmounts SPI NAND flash from the FATFS file system.

- **Prototype**

```
ql_errcode_nand_flash_e ql_nand_flash_umount(void)
```

- **Parameter**

None

- **Return Value**

See ***Chapter 3.2.3.1.1*** for details.

3.2.3.3.ql_nand_flash_mount

This function mounts SPI NAND flash to the FATFS file system.

- **Prototype**

```
ql_errcode_nand_flash_e ql_nand_flash_mount(void)
```

- **Parameter**

None

- **Return Value**

See ***Chapter 3.2.3.1.1*** for details.

3.2.3.4.ql_nand_get_hw_info

This function gets hardware information of SPI NAND flash.

- **Prototype**

```
ql_errcode_nand_flash_e ql_nand_get_hw_info(ql_nand_hw_info_t *info)
```

- **Parameter**

info:
[Out] Hardware information of SPI NAND flash. See ***Chapter 3.2.3.1.1*** for details.

- **Return Value**

See ***Chapter 3.2.3.1.1*** for details.

3.2.3.4.1. ql_nand_hw_info_t

The structure of hardware information of SPI NAND flash:

```
typedef struct
{
    uint32_t mid;
    uint32_t blknum;
    uint32_t blksize;
} ql_nand_hw_info_t
```

- **Parameter**

Type	Parameter	Description
uint32_t	*mid*	Manufacturer ID
uint32_t	*blknum*	Block count
uint32_t	*blksize*	Block size

4 Example

This chapter explains how to use the above APIs at the application layer for development and simple debugging of general (4-wire) SPI NAND flash and SPI NAND flash file system mounting.

NOTE

When you replace NOR flash on TE-A with NAND flash for testing, if you are using Standard SPI or Dual SPI to drive NAND flash, consider the following:

1. SIO2 and SIO3 pins need to be disconnected;
2. If SIO2 and SIO3 are connected, they should be set as push–pull output.

4.1. Development Example

4.1.1. General SPI NAND Flash API

\components\ql-application\spi_nand_flash\spi_nand_flash_demo.c, the demo file for general SPI NAND flash API, is provided in QuecOpen CSDK of the module. It includes:

- General SPI NAND flash initialization
- Reading and writing the value of status register
- Reading and writing the main data area and spare area of a page
- Erasing a block
- Reading NAND flash JEDEC ID
- Resetting NAND flash

The entry function is *ql_spi_nand_flash_demo_init()*, as shown below:

```
QlOSStatus ql_spi_nand_flash_demo_init(void)
{
    QlOSStatus err = QL_OSI_SUCCESS;

    err = ql_rtos_task_create(&spi_nand_flash_demo_task, SPI_NAND_FLASH_DEMO_TASK_STACK_SIZE, SPI_NAND_FLASH_DEMO_TASK_PRIO, "ql_spi_nand", ql_spi_nand_flash_demo_task_pthread, NULL, SPI_NAND_FLASH_DEMO_TASK_EVENT_CNT);
    if(err != QL_OSI_SUCCESS)
    {
        QL_SPI_NAND_LOG("demo_task created failed");
        return err;
    }

    return err;
}
```

Figure 4: Entry Function: *ql_spi_nand_flash_demo_init()*

As shown below, the above example is disabled by default in *ql_init_demo_thread*. Uncomment

ql_spi_nand_flash_demo_init() if you need to set the example auto-start.

```
#ifdef QL_APP_FEATURE_SPI_NAND_FLASH
    //ql_spi_nand_flash_demo_init();
#endif
```

Figure 5: General SPI NAND Flash API Example Disabled by Default

4.1.2. SPI NAND Flash File System Mount API

\components\ql-application\fs_nand_flash\fs_nand_flash_demo.c, the demo file for SPI NAND flash file system mount API, is provided in QuecOpen CSDK of the module. It includes:

- SPI NAND flash initialization
- FATFS file system mounting
- SPI NAND flash formatting
- Reading and writing through file system
- Unmounting FATFS file system

The entry function is *ql_fs_nand_flash_demo_init()*, as shown below:

```
QlOSStatus ql_fs_nand_flash_demo_init(void)
{
    QlOSStatus err = QL_OSI_SUCCESS;

    err = ql_rtos_task_create(&fs_nand_flash_demo_task, FS_NAND_FLASH_DEMO_TASK_STACK_SIZE, FS_NAND_FLASH_DEMO_TASK_PRIO, "ql_nand_flash", ql_fs_nand_flash_demo_task_thread, NULL, FS_NAND_FLASH_DEMO_TASK_EVENT_CNT);
    if(err != QL_OSI_SUCCESS)
    {
        QL_NAND_FLASH_LOG("demo_task created failed");
        return err;
    }

    return err;
}
```

Figure 6: Entry Function: *ql_fs_nand_flash_demo_init()*

As shown below, the above example is disabled by default in *ql_init_demo_thread*. Uncomment *ql_fs_nand_flash_demo_init()* if you need to set the example auto-start.

```
#ifdef QL_APP_FEATURE_FS_NAND_FLASH
    //ql_fs_nand_flash_demo_init();
#endif
```

Figure 7: SPI NAND Flash File System Mount API Example Disabled by Default

4.2. Feature Debugging

4.2.1. General SPI NAND Flash API

Before debugging the SPI NAND flash feature, it is necessary to connect a NAND flash chip to LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to a PC with a USB cable. USB AP Log Port and USB DIAG Port primarily display system debugging information. You can view the example through USB AP Log Port or USB DIAG Port after capturing the log with cooltools or ArmLogel. For details about log capture, see ***documents [3]*** and ***[4]***.

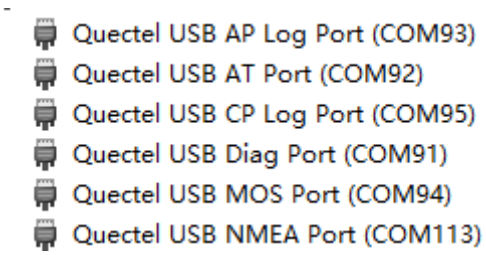

Figure 8: Ports in Device Manager (EC600U Series)

Figure 9: Ports in Device Manager (ECx00G Family/EG800G Series)

NOTE

For EC600U series module, cooltools is used to capture logs through USB AP Log Port; see ***document [4]*** for details on log capture. For ECx00G family and EG800G series modules, ArmLogel is used to capture logs through USB DIAG Port; see ***document [3]*** for details on log capture.

ql_spi_nand_flash_demo_init() runs automatically after the module is turned on. Determine whether the read and write operations are successful by checking the consistency of the read and written data in main data area and spare area of general SPI NAND flash page. Read data after erasing a NAND flash block, which can be identified as successful if all the read data are 'FF'.

In case of a failed execution, you can view the failure cause in the corresponding log.

Index	Received	Tick	Level	Description
71	09:58:44.532	41192	QOPN/I	[ql_SPI][ql_spi_write, 494] write sucess
72	09:58:44.532	41194	QOPN/I	[ql_SPI_NAND][ql_spi_nand_wait_write_finish, 244] status=2
73	09:58:44.532	41194	QOPN/I	[ql_SPI][ql_spi_write, 494] write sucess
74	09:58:44.532	41194	QOPN/I	[ql_SPI_NAND][ql_spi_nand_wait_write_finish, 244] status=0
99	09:58:44.563	41391	QOPN/I	[ql_SPI][ql_spi_write, 494] write sucess
100	09:58:44.563	41391	QOPN/I	[ql_SPI][ql_spi_write, 494] write sucess
103	09:58:44.579	41683	QOPN/I	[ql_SPI_NAND][ql_spi_nand_read_devid, 534] mid =efbf22
104	09:58:44.579	41684	QOPN/I	[ql_SPI_NAND_DEMO][ql_spi_nand_flash_demo_task_pthread, 186] sr1 = 0
105	09:58:44.579	41684	QOPN/I	[ql_SPI_NAND_DEMO][ql_spi_nand_flash_demo_task_pthread, 195] sr2 = 18
106	09:58:44.579	41684	QOPN/I	[ql_SPI_NAND_DEMO][ql_spi_nand_flash_demo_task_pthread, 204] sr3 = 0
107	09:58:44.579	41685	QOPN/I	[ql_SPI_NAND_DEMO][ql_spi_nand_flash_demo_task_pthread, 213] sr4 = 0
108	09:58:44.579	41685	QOPN/I	[ql_SPI_NAND_DEMO][ql_spi_nand_flash_demo_task_pthread, 222] a2 = 0xaaaaaaaa

Figure 10: Log Information (General SPI NAND Flash)

4.2.2. SPI NAND Flash File System Mount API

Before debugging the SPI NAND flash file system mount feature, it is necessary to connect a NAND flash chip to LTE OPEN EVB.

Download the compiled firmware to the module and connect the USB port of the LTE OPEN EVB to a PC with a USB cable. USB AP Log Port and USB DIAG Port primarily display system debugging information. You can view the example through USB AP Log Port or USB DIAG Port after capturing the log with cooltools or ArmLogel. For details about log capture, see ***documents [3]*** and ***[4]***. For ports in device manager of the module, see ***Figure 8*** and ***Figure 9***.

NOTE

Select one SPI mode (that is, Standard SPI, Dual SPI, or Quad SPI, as shown below) using the *QL_SPI_TPYE* macros in *fs_nand_flash_demo.c*. After that, the demo file will adjust the initialization process according to the selected SPI mode.

```
#define QL_STANDARD_SPI  0    //Standard SPI: SCLK, CS#, SI, SO
#define QL_QUAD_SPI      1    //Quad SPI: SCLK, CS#, SIO0, SIO1, SIO2, SIO3
#define QL_DUAL_SPI      2    //Dual SPI: SCLK, CS#, SIO0, SIO1

#define QL_SPI_TPYE  QL_QUAD_SPI
```

ql_fs_nand_flash_demo_init() runs automatically after the module is turned on. You can view the SPI NAND flash file system mounting information through logs. Log port debugging information is shown in the figure below.

```
[14:53:33.240]  [19403] [ 1020] [                ] [      ] [ ] QOPN/I : ql_spi_nand_read_devid_ex 592 mid[0] =efbf
[14:53:33.240]  [19404] [ 1021] [                ] [      ] [ ] QOPN/I : ql_spi_nand_read_devid_ex 592 mid[1] =efbf22
[14:53:33.240]  [19413] [ 1022] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=0, block=0
[14:53:33.240]  [19549] [ 1023] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=0, used_bit=64
[14:53:33.240]  [19553] [ 1024] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=1, block=1
[14:53:33.240]  [19685] [ 1025] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=1, used_bit=62
[14:53:33.240]  [19689] [ 1026] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=2, block=2
[14:53:33.240]  [19820] [ 1027] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=2, used_bit=64
[14:53:33.240]  [19824] [ 1028] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=3, block=3
[14:53:33.240]  [19952] [ 1029] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=3, used_bit=64
[14:53:33.240]  [19957] [ 1030] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:599 position=4, block=4
[14:53:33.299]  [20085] [ 1031] [                ] [      ] [ ] QUEC/I : ql_ftl_init_cache_block:665 position=4, used_bit=64
[14:53:33.524]  [24717] [ 1032] [                ] [      ] [ ] QUEC/I : ql_ftl_table_create:862 nand read phy total=2048, bad_blocknum=0, good_blocknum=2048
[14:53:33.588]  [24717] [ 1033] [                ] [      ] [ ] QUEC/I : ql_ftl_table_create:863 data_block_num=1904, free_blocknum=139, cache_blocknum=5
[14:53:33.741]  [27484] [ 1034] [                ] [      ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b-:65,new-b:2046
[14:53:33.741]  [27484] [ 1035] [                ] [      ] [ ] QUEC/I : ql_ftl_mark_cache_block_position:704 mark block_position=0x1
[14:53:33.741]  [27493] [ 1036] [                ] [      ] [ ] QUEC/I : ql_ftl_exchange_cache_data:1345 exchange cache position=1, count=1
[14:53:33.741]  [27609] [ 1037] [                ] [      ] [ ] QUEC/I : ql_ftl_exchange_cache_data:1368 cache old-b-:1,new-b:2047
[14:53:33.741]  [27611] [ 1038] [                ] [      ] [ ] QUEC/I : ql_mount_nand_flash:288 ql_mount_nand_flash: block count: 121856, block size: 2048
[14:53:33.741]  [27676] [ 1058] [                ] [      ] [ ] QUEC/I : ql_fs_mount_nand_flash:328 mount nand flash ret: 1
[14:53:33.741]  [27677] [ 1059] [                ] [      ] [ ] QOPN/I : ql_fs_nand_flash_demo_task_thread 170 mid=efbf22, total_size = 249561088
[14:53:33.741]  [27683] [ 1060] [                ] [      ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/4096 count/1
[14:53:33.741]  [27704] [ 1061] [                ] [      ] [ ] QOPN/I : ql_mkdir 673 Dir is already exist
[14:53:33.741]  [27704] [ 1062] [                ] [      ] [ ] QOPN/I : ql_fs_nand_flash_demo_task_thread 182 dir exist, not create
[14:53:33.741]  [27707] [ 1063] [                ] [      ] [ ] QOPN/I : ql_get_file_info 690 get node count failed, or no file in list
[14:53:33.741]  [27712] [ 1064] [                ] [      ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/4097 count/1
[14:53:33.741]  [27735] [ 1065] [                ] [      ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/4097 count/1
[14:53:33.741]  [28067] [ 1066] [                ] [      ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b-:2041,new-b:2045
[14:53:33.741]  [28068] [ 1067] [                ] [      ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/3620 count/1
[14:53:33.741]  [28088] [ 1068] [                ] [      ] [ ] QUEC/I : ql_nand_block_erase:101 nand erase block start num = 4098, count = 1
[14:53:33.750]  [28089] [ 1069] [                ] [      ] [ ] QUEC/I : ql_ftl_erase_continue_page:2335 head_num: 1, tail_num:0, mid_num:0, erase_block_num:0
[14:53:33.750]  [28305] [ 1070] [                ] [      ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b-:2045,new-b:2044
[14:53:33.814]  [28306] [ 1071] [                ] [      ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/3620 count/1
[14:53:33.838]  [29596] [ 1072] [                ] [      ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b-:2042,new-b:2043
[14:53:33.838]  [29596] [ 1073] [                ] [      ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/3858 count/1
[14:53:33.838]  [29619] [ 1074] [                ] [      ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/4097 count/1
[14:53:33.838]  [29640] [ 1075] [                ] [      ] [ ] QOPN/I : ql_fopen 169 open file EXNAND:/test_dir/test_file1.txt, fd 4099
[14:53:33.838]  [29649] [ 1076] [                ] [      ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/3620 count/1
[14:53:33.838]  [29672] [ 1078] [                ] [      ] [ ] QOPN/I : ql_fs_nand_flash_demo_task_thread 222 file read result is 0123456789ABCDEFGHIGKLMNOPQRSTUVWXYZ
[14:53:33.838]  [29674] [ 1079] [                ] [      ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1e97c) nr/4098 count/1
[14:53:33.892]  [29698] [ 1080] [                ] [      ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/3620 count/1
[14:53:33.900]  [30893] [ 1081] [                ] [      ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b-:2043,new-b:2039
[14:53:33.900]  [30893] [ 1082] [                ] [      ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/3858 count/1
[14:53:33.900]  [30921] [ 1083] [                ] [      ] [ ] QUEC/I : ql_nand_block_read:146 read (0x80c1deb4) nr/4097 count/1
[14:53:33.923]  [30941] [ 1084] [                ] [      ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/4097 count/1
[14:53:33.923]  [31179] [ 1085] [                ] [      ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b-:2044,new-b:2038
[14:53:33.967]  [31181] [ 1086] [                ] [      ] [ ] QUEC/I : ql_nand_block_write:175 write (0x80c1deb4) nr/64 count/1
[14:53:33.967]  [31515] [ 1087] [                ] [      ] [ ] QUEC/I : ql_ftl_exchange_data_free_node:1885 change old-b-:2040,new-b:2037
[14:53:33.967]  [31516] [ 1088] [                ] [      ] [ ] QOPN/I : ql_fclose 264 close file fd 4099
[14:53:33.967]  [31516] [ 1089] [                ] [      ] [ ] QOPN/I : ql_fs_nand_flash_demo_task_thread 242 ql_rtos_task_delete
[14:53:34.013]  [31518] [ 1090] [                ] [      ] [ ] QOPN/I : ql_rtos_task_delete 191 remove task 80C03CA8 ok
```

Figure 11: Log Information (SPI NAND Flash File System Mount)

5 Appendix References

Table 4: Related Documents

Document Name	
[1]	Quectel_ECx00G&EC600U&EG800G_Series_QuecOpen(SDK)_CSDK_Quick_Start_Guide
[2]	Quectel_EC200D&ECx00G&EC600U&EG800G_Series_QuecOpen(SDK)_GPIO_API_Reference_Manual
[3]	Quectel_ECx00G&EG800G_Series_QuecOpen(SDK)_Log_Capture_Guide
[4]	Quectel_EC600U_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 5: Terms and Abbreviations

Abbreviation	Description
API	Application Programming Interface
CS	Chip Select
DI	Data Input
DMA	Direct Memory Access
ECC	Error Correcting Code
EVB	Evaluation Board
FIFO	First In First Out
GPIO	General-Purpose Input/Output
ID	Identifier
IoT	Internet of Things
JEDEC	Joint Electron Device Engineering Council

LTE	Long-Term Evolution
NAND	NON-AND
PC	Personal Computer
RTOS	Real-Time Operating System
SD	Secure Digital Card
SDK	Software Development Kit
SPI	Serial Peripheral Interface
USB	Universal Serial Bus